

Reviewer Pack — Minimal Local Indeterminacy of Matroid Representations and C...

Entry 85beb8f8b8a8 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-03 18:48:31 UTC

Minimal Local Indeterminacy of Matroid Representations and Communication Complexity Rank

Entry ID: 85beb8f8b8a8 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-03 18:48:31 UTC

1. Conjecture

Field A (mathematical branch): Combinatorial Geometry (Matroid Theory) **Field B** (complexity object): Communication Complexity

Statement:

The minimal local indeterminacy of a matroid representation is linearly correlated with its communication complexity rank, such that the rank $r(M) = \Theta(\Gamma(M))$, where M is a given matroid and $\Gamma(M)$ is its associated communication complexity rank.

Rationale (proposer's reasoning):

Matroid theory provides a framework for studying combinatorial structures through representations. Local indeterminacy measures the lack of uniqueness in these representations. This conjecture suggests that this property might be related to communication complexity, which measures the amount of communication needed for two parties to reach agreement. If true, it could offer new insights into both fields.

Taxonomy category: MATROIDS_X_COMMUNICATION_COMPLEXITY_RANK (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): d8b86b7236f3d552

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the correlation coefficient between minimal local indeterminacy ($\alpha(M)$) and communication complexity rank ($r(M)$) for all matroids M , computed across 30 different seeds, is ≥ 0.8 AND the mean absolute difference between $\alpha(M)$ and $r(M)$ divided by $n \leq 3$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	1.00	SAFE	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 2 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "minimal local indeterminacy" AND "matroid representation" AND "communication complexity rank" - "matroid theory" AND "communication complexity" AND "indeterminacy" - " $\Gamma(M)$ " AND " $r(M)=\theta(\Gamma(M))$ " AND "matroid theory"

Top relevant hits considered: - [s2:10.26565/2312-4334-2025-1-44] Anisotropic Cosmological Model in a Modified Theory of Gravitation - [s2:10.1021/acs.joc.4c02653] Diastereospecific Synthesis of Vicinally Substituted 2-Oxazolidinones via Oxidative Rearrangement of α,β -Unsaturated γ -L

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction
from itertools import combinations

def gaussian_elimination(matrix):
    n = len(matrix)
    for i in range(n):
        # Find pivot row
        max_row = i
        for j in range(i+1, n):
            if abs(matrix[j][i]) > abs(matrix[max_row][i]):
                max_row = j
        matrix[i], matrix[max_row] = matrix[max_row], matrix[i]

        # Eliminate below pivot
        for j in range(i+1, n):
            factor = Fraction(matrix[j][i], matrix[i][i])
            for k in range(n + 1):
                if k < i:
                    matrix[j][k] -= factor * matrix[i][k]
                elif k == i:
                    matrix[j][k] = Fraction(0)
                else:
                    matrix[j][k] -= factor * matrix[i][k]

def rank(matrix):
    n = len(matrix)
    augmented_matrix = [row + [1 if i == j else 0 for j in range(n)] for i, row in enumerate(matrix)]
    gaussian_elimination(augmented_matrix)
```

```

    return sum(1 for row in augmented_matrix if any(row[j] != Fraction(0) for j in range(n)) and a

def min_local_indeterminacy(matroid):
    n = len(matroid)
    matrix = [[Fraction(0)] * (n + 1) for _ in range(n)]
    for i in range(n):
        for j in range(i+1, n):
            if matroid[i][j] == 1:
                matrix[i][j] = Fraction(1)
                matrix[j][i] = Fraction(1)

    gaussian_elimination(matrix)
    rank_matrix = rank(matrix)
    return rank_matrix

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n_max = 40
    instances_tested = 0
    alpha_values = []
    r_values = []

    for n in range(5, n_max + 1):
        for _ in range(6): # Ensure at least 30 instances per seed (6 instances per size)
            matroid = [[random.choice([0, 1]) for _ in range(n)] for _ in range(n)]
            alpha = min_local_indeterminacy(matroid)
            r = rank(matroid)
            alpha_values.append(alpha)
            r_values.append(r)
            instances_tested += 1

    correlation_coefficient = sum((alpha - mean_alpha) * (r - mean_r) for alpha, r in zip(alpha_values, r_values))
    mean_alpha = sum(alpha_values) / len(alpha_values)
    mean_r = sum(r_values) / len(r_values)
    max_diff_over_n = max(abs(alpha - r) / n for alpha, r in zip(alpha_values, r_values))

    conjecture_holds = correlation_coefficient >= 0.8 and max_diff_over_n <= 3
    counterexample = "" if conjecture_holds else f"Correlation: {correlation_coefficient}, Max diff: {max_diff_over_n}"

    return {
        "metric_name": "Communication Complexity Rank",
        "metric_value": correlation_coefficient,
        "instances_tested": instances_tested,
        "n_max": n_max,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(seed) for seed in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
    results = []

    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")

```

```

results.append(trial_result)

mean_metric_value = sum(result["metric_value"] for result in results) / len(results)
std_metric_value = math.sqrt(sum((result["metric_value"] - mean_metric_value) ** 2 for result
support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

if all(result["conjecture_holds"] for result in results):
    print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
elif support_fraction >= 0.8:
    print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
else:
    first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"Correlation too low\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_7077ebad.py", line 96, in <module>
    trial_result = run_trial(seed)
                    ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_7077ebad.py", line 67, in run_trial
    alpha = min_local_indeterminacy(matroid)
             ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_7077ebad.py", line 53, in min_local_indeterminacy
    gaussian_elimination(matrix)
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_7077ebad.py", line 29, in gaussian_elimination
    factor = Fraction(matrix[j][i], matrix[i][i])
              ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/usr/lib/python3.12/fractions.py", line 281, in __new__
    raise ZeroDivisionError('Fraction(%s, 0)' % numerator)
ZeroDivisionError: Fraction(0, 0)

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed due to a division by zero error, preventing the computation of the required correlation coefficient and mean absolute difference. | next: Investigate the cause of the division by zero error in the gaussian_elimination function and fix it before re-running the test.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	21358
2	preregistration	ollama_remote	glm4:latest	0	0	14247
3	novelty	ollama_remote	glm4:latest	0	0	16009
4	novelty	ollama_remote	glm4:latest	0	0	12670
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	22754
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17970
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	20958
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18639
9	judge	ollama_remote	glm4:latest	0	0	10726

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 155330 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/85beb8f8b8a8.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/85beb8f8b8a8.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/85beb8f8b8a8.tar.gz` (if generated)